

Introduction Parallel Structures

We define four “styles” of parallel computing:

SISD Single instruction, single data. Standard computers.

There is one CPU that can process one instruction stream on one set of data.

SIMD Single instruction, multiple data. Vector and array processors (supercomputers). In these, a single instruction stream is applied simultaneously to different data streams.

MISD Multiple instruction, multiple data. Not in commercial use.

MIMD Multiple instruction, multiple data. This is where multiple processor systems fit. Each CPU processes a separate instruction stream (program) on separate data. They may (or may not) be working together on a common task.

Multiple processor systems (MIMD)

Multiple processor systems are probably easiest to understand, due to their similarity to multiprogramming (multitasking) environments.

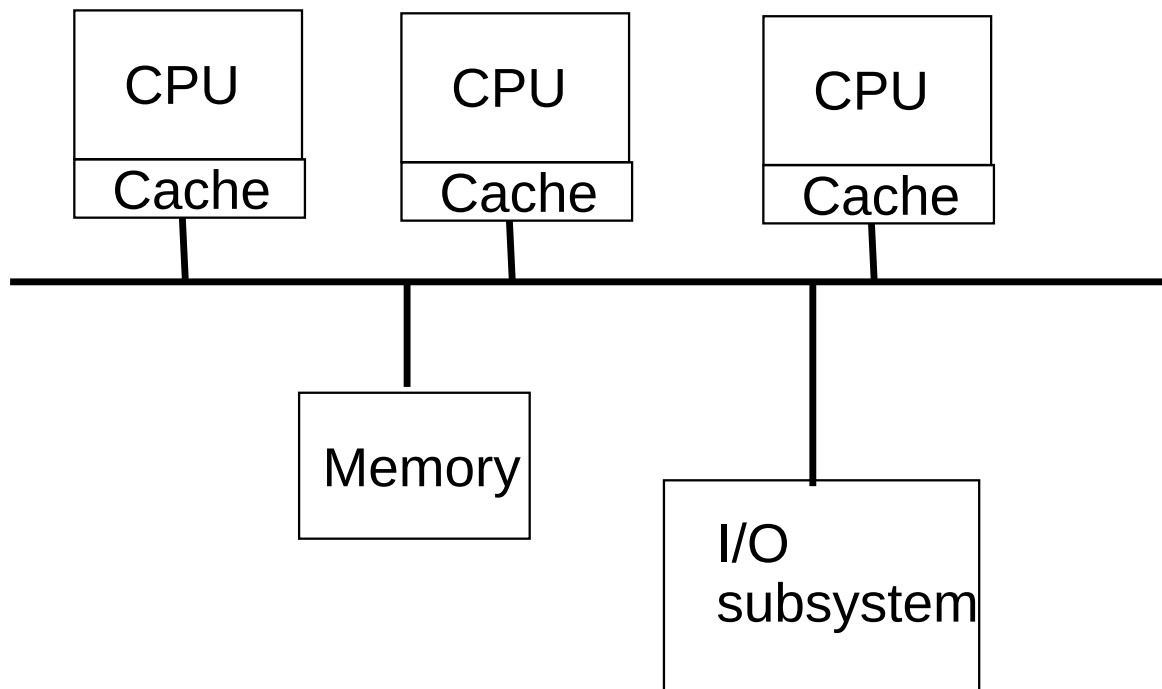
In a multi-tasking operating system, we allowed several processes to be **active** simultaneously, while insisting that only one was **executing**. With multi-processor systems, we can have distribute active processes across different CPUs, so that several can be executing simultaneously.

Symmetric multi processor (SMP) systems

In an SMP system, we assume that all processors are identical, with approximately identical access times to memory. In theory, this process can (and should) be transparent to the user.

In SMP systems, processes executing on different processors can communicate with each other (and with the operating system) through shared memory resources, therefore some topology must exist for interconnection.

The most common interconnection is through a **shared memory bus**



The bus must provide:

Addressing It must be possible to distinguish the source and destination modules for each bus transfer.

Arbitration There must be some mechanism for deciding access to the bus. This is usually done through some Request/Grant logic. Common algorithms include round-robin, first-come-first-served, and priority schemes.

Time sharing CPUs that require access to the bus when it is held by another unit will have to stall until the bus is available. This is a potential performance bottleneck.

Since the single shared bus is a performance bottleneck, each processing unit is equipped with its own local cache.

This, however, carries its own potential problem, known as the **cache coherence problem** (which we have already mentioned before). In a situation when two processors have local cached copies of the same cache line, a write by one must invalidate the data belonging to the other.

This problem can fundamentally be solved in software or in hardware.

Software solutions for cache coherence

Software solutions to cache coherence are based on the fact that if no two processes share a cache line, the problem does not exist. Therefore, if compilers flag some variables as “not cacheable” (ie: they have to reside in main memory, either permanently, or during some “dangerous” periods), cache coherence is guaranteed.

Since this is a compile time approach, it will not lead to an run-time, except when necessary. However, it is restrictive, and requires a great deal of *a priori* analysis of the programs to be run.

Hardware solutions for cache coherence

When we first looked at cache coherence, we proposed the “write-through” approach, in which all writes went through the cache to the main memory. This will solve the problem, but it is undesirable, because:

- It will lead to increased bus traffic.
- It can lead to **unnecessary** bus traffic. If no two caches share the same cache line, then write-back policies will work fine.

What we need is the ability to apply write-through *only when necessary*.

There are two fundamental approaches to hardware cache coherence:

In a **directory protocol**, a central controller keeps track of locations, and number of copies of cache lines. If a particular cache wants to write to a given cache line, it will request **exclusive (write) access** to that cache line. When the directory grants exclusive access to a given cache line, all other caches that have non-exclusive access must invalidate their copies.

If a cache wants a cache line and some other cache has exclusive access, the directory will force it to write back before allowing a new read.

The other approaches to cache coherence are called **snooping protocols**. In these types of systems, each caches must know (by monitoring bus activity) if it shares a cache with others. If it does, then it must somehow keep others that share the line up-to-date on its status.

Write-broadcast (write-update) schemes are essentially the multi-cache equivalent of write through. Each cache distributes each write to the entire system.

Write-invalidate schemes allow only one writer at a time. A cache that intends to write a word will inform others of its intent. They will invalidate their copies, leading to misses if processors try to read there.

In the standard write-invalidate scheme, called the MESI protocol, cache lines can be in one of four states:

Modified This cache has modified this cache line. This is the only valid copy of this line.

Exclusive This line is present only in this cache. This line is not dirty (matches main memory).

Shared This line is present in this cache and at least one other. This line is not dirty.

Invalid This line contains invalid data and should lead to a miss if accessed.

Operating system duties in SMP systems

Reentrancy Operating system routines can be called simultaneously by different processors. Any OS data must therefore be carefully managed.

Synchronization Processes that share data and resources must be synchronized. Some buses support this through the **read-modify-write** instruction. In other cases, software solutions are required.

Memory management There is a possible “page coherence” problem that is equivalent to the “cache coherence” problem. Memory management across processors must enforce consistency.

Reliability The failure of one processor should not bring down the system.